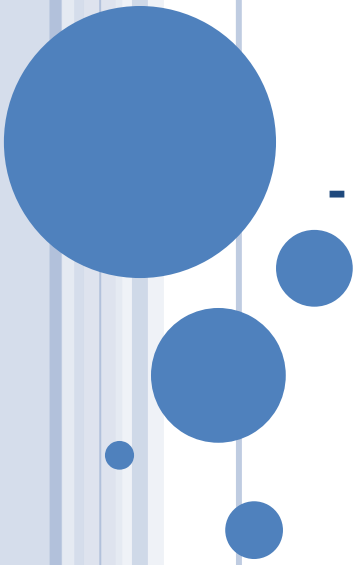
MPI – *MESSAGE PASSING INTERFACE*



- Parte 1 -

SUMÁRIO

- Introdução
- Funções Básicas – parte I
- O que é uma Mensagem?
- MPI *Datatype*
- Comunicação Ponto-a-Ponto
- Funções Básicas – parte II



CONSIDERAÇÕES FINAIS

- Programa em C deve conter

```
#include <mpi.h>
```

- Para compilar:

```
mpicc –o arq arq.c
```

- Para executar
  - MPICH 1

```
mpirun –np 2 arq
```

  - MPICH 2

```
mpiexec –np 2 arq
```

```
sudo apt-get install libopenmpi-dev
openmpi-bin openmpi-doc
```



INTRODUÇÃO

- O padrão foi desenvolvido em 1993 por
  - indústrias (Cray, SGI, HP, Mach, NEC, Intel, IBM)
  - universidades
  - laboratórios de pesquisa
- Várias implementações
  - mpich (ANL e MSU)
  - CHIMP (Univ. Edinburgh)
  - LAM (Ohio Supercomputers)



INTRODUÇÃO (2)

- SPMD – *Single Program, Multiple Data*
- Mestre – escravo

```
if (mestre) { ... }
else {...}
```

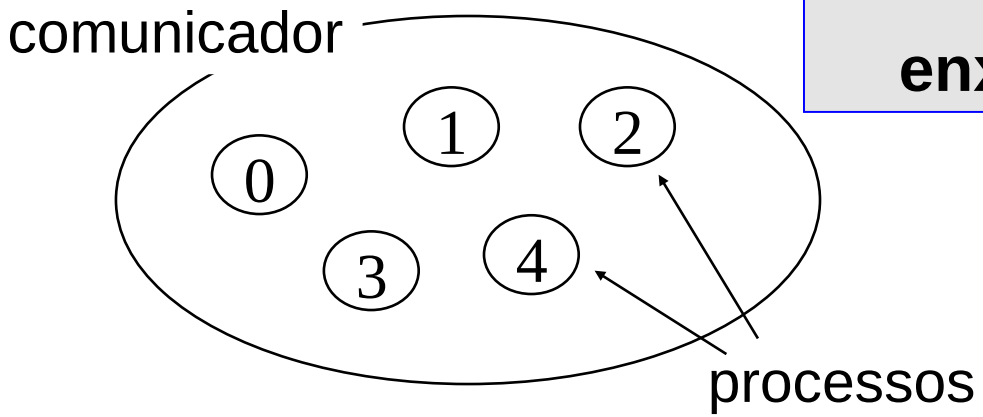
- Único arquivo executável



INTRODUÇÃO (3)

- Identificadores da biblioteca começam com `MPI_`
- As mensagens somente podem ser transmitidas dentro de um comunicador

Conjunto de processos que conseguem enxergar uns aos outros



INTRODUÇÃO (4)

- Cada processo em um comunicador é identificado por um número [0, n-1], onde n é o número total de processos
  - *Rank* ou identificador
- As mensagens devem identificar
  - o comunicador
  - o processo destinatário, por meio do identificador (*rank*)
- Quando um processo chama a função de inicialização do MPI, ele passa a pertencer ao comunicador padrão, ao qual todos os processos pertencem
  - `MPI_COMM_WORLD`



FUNÇÕES BÁSICAS – PARTE I

- `MPI_Init`
- `MPI_Finalize`
- `MPI_Comm_size`
- `MPI_Comm_rank`



MPI_INIT

- A primeira rotina em MPI que deve ser chamada (somente uma vez!) é a `MPI_INIT`
- Essa chamada possibilita à biblioteca MPI fazer as inicializações necessárias, que podem variar entre implementações

```
int MPI_Init(int *argc, char ***argv);
```

- `MPI_Init` define um comunicador, construído pelo grupo de processos que chamam `MPI_Init`. `MPI_COMM_WORLD` é o comunicador padrão.



MPI_FINALIZE

- Depois do programa terminar de usar o MPI, ele deve avisar a biblioteca
  - Permite ao MPI remover mensagens pendentes, liberar estruturas de dados...
- Nenhuma outra função do MPI pode ser invocada após essa chamada

```
int MPI_Finalize(void)
```



MPI_COMM_SIZE

- Um processo MPI pode consultar o comunicador para obter informações sobre o grupo, como...

- `MPI_Comm_size` devolve número de processo que fazem parte do comunicador

```
int MPI_Comm_size(MPI_Comm comm,
                  int*     number_of_process)
```



MPI_COMM_RANK

- e …

- MPI_Comm_rank devolve o identificador (número) do processo no comunicador

```
int MPI_Comm_rank(MPI_Comm comm,int* my_rank)
```



EXEMPLO

```
#include <stdio.h>
#include <mpi.h>
int main(int argc, char* argv[]){
   int p, id;
   MPI_Init (&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &id);
   MPI_Comm_size(MPI_COMM_WORLD,&p);
   printf("- Hello World!!!, from %d\n", id);
   MPI_Finalize ();
   return 0;
}
```



O QUE É UMA MENSAGEM?

- Uma mensagem é um vetor de elementos de um determinado tipo de dado do MPI (*datatype* MPI)

MPI_INT	**MPI_INT**	**MPI_INT**	**MPI_INT**	**MPI_INT**



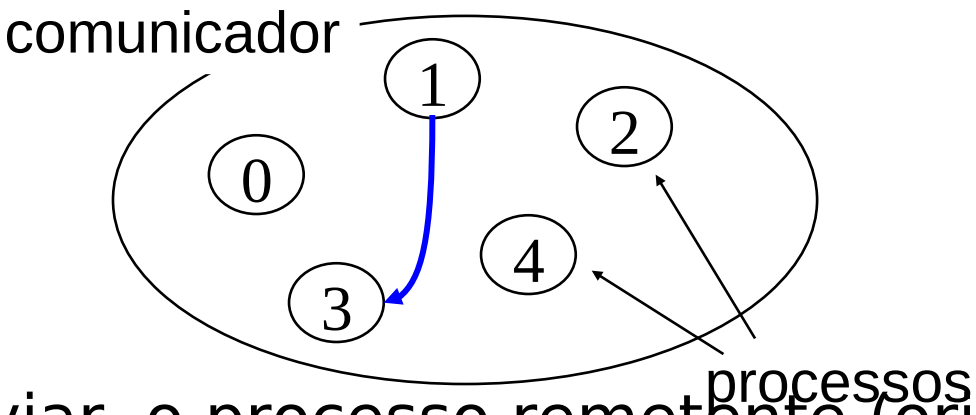
MPI *DATATYPE*

- **Todas as mensagens em MPI são tipadas**
  - **O *send* e o *receive* devem especificar o tipo de mensagem**

MPI Datatype	C datatype
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	
MPI_PACKED	

COMUNICAÇÃO PONTO-A-PONTO

- Entre dois processos



- Para enviar, o processo remetente (origem) identifica o *rank* do processo destinatário (destino)

- O processo destino também deve identificar de quem receberá a mensagem.

COMUNICAÇÃO PONTO-A-PONTO (2)

- Há 4 modos de comunicação
  - *Standard*
    - o send é completo, independentemente se a mensagem foi recebida
  - *Synchronous*
    - o *send* somente estará completo quando o *receive* estiver terminado
    - garantia que o processo destino recebeu a mensagem
  - *Buffered*
    - o send está sempre completo
    - uso de buffers para armazenar as mensagens até serem enviadas
    - o programador é responsável pela manutenção dos *buffers*
  - *Ready*
    - *“a mensagem será recebida se o destinatário desejar”*

COMUNICAÇÃO PONTO-A-PONTO (3)

- Todas as 4 formas podem ser
  - bloqueantes (a rotina precisa ter sido terminada)

  ou
  - não-bloqueantes (devem ser testado o término com `MPI_TEST`, `MPI_WAIT`)

	Blocking form
Standard send	MPI_SEND
Synchronous send	MPI_SSEND
Buffered send	MPI_BSEND
Ready send	MPI_RSEND
Receive	MPI_RECV



FUNÇÕES BÁSICAS – PARTE II

- `MPI_Send`
- `MPI_Recv`



MPI_SEND

- *Standard Send*
  - Envia a mensagem sem a garantia desta ter chegado ao seu destino

```
int MPI_Send(void*          message,
             int            count,
             MPI_Datatype       datatype,
             int          dest,
             int          tag,
             MPI_Comm       comm)
```

- Onde:

MPI_SEND (2)

- `message` é o endereço do dado a ser enviado
- `count` é o número de elementos do tipo *MPI_Datatype* que `message` contém
- `datatype` é o *MPI_Datatype*
- `dest` é o *rank* do processo destinatário
- `tag` é usado para distinguir diferentes tipos de mensagens. O programador pode usá-los para ordenar as mensagens.
- `comm` é o comunicador



MPI_RECV

- Bloqueante

```
int MPI_Recv(void*          message,
             int            count,
             MPI_Datatype   datatype,
             int            source,
             int            tag,
             MPI_Comm       comm,
             MPI_Status     status)
```

- Onde:



MPI_RECV (2)

- `message` é o endereço onde o dado será armazenado
- `count` é o número de elementos do tipo *MPI_Datatype* que serão recebidos
- `datatype` é o *MPI_Datatype*. Deve ser compatível com o especificado no *send*
- `source` é o *rank* do processo remetente
- `tag` é usado para definir o recebimento de uma mensagem com determinado tag. Pode-se usar `MPI_ANY_TAG`.
- `comm` é o comunicador
- `status` armazena informações sobre o recebimento da mensagem.

EXEMPLO

```
#include <mpi.h>
#include <stdio.h>
  main(int argc, char* argv[]){
    int p, id, source, dest, tag=1;
    char message[100];
    MPI_Status status;
    MPI_Init (&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD,&p);
    ...
```



EXEMPLO – CONT.

```
  if (id != 0){
    sprintf(message, “Saudações do processo %d ”, id);   dest =
  0;
    MPI_Send(message, strlen(message)+1, MPI_CHAR, dest,
                                          tag, MPI_COMM_WORLD);
  } else
    for(source=1; source<p; source++){
        MPI_Recv(message, 100, MPI_CHAR, source, tag,
                                      MPI_COMM_WORLD, &status);
        printf(“Processo %d: %s – recebida do Processo %d\n”,
  id, message, source);
    }
  MPI_Finalize ();
  return 0;
}
```

CONSIDERAÇÕES FINAIS

- Programa em C deve conter

```
#include <mpi.h>
```

- Para compilar:

```
mpicc –o arq arq.c
```

- Para executar
  - MPICH 1

```
mpirun –np 2 arq
```

  - MPICH 2

```
mpiexec –np 2 arq
```



BIBLIOGRAFIA

PACHECO, Peter. Parallel Programming with MPI. San Francisco: Morgan Kaufmann, 1997.

MPI: A Message-Passing Interface Standard. Disponível em http://www.mpi-forum.org/docs/mpi-11-html/mpi-report.html

.

